T3000 Engineering

Debugging the webview · built Nov 13, 2026

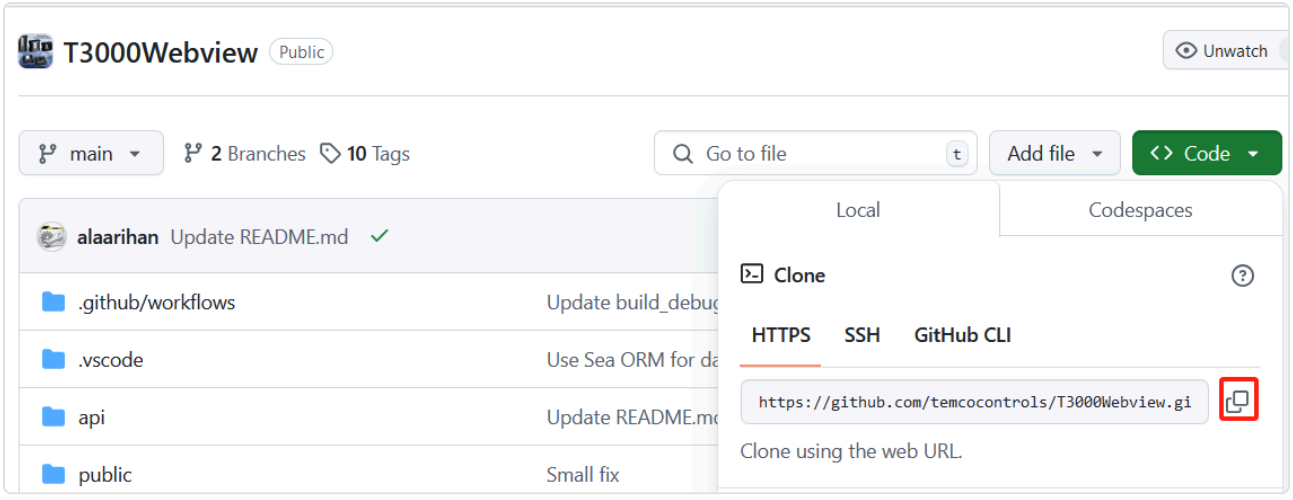
Debugging the webview

The webview is a separate process from the C++ shell, so debugging it means attaching to the right one. This is the setup, screen by screen.

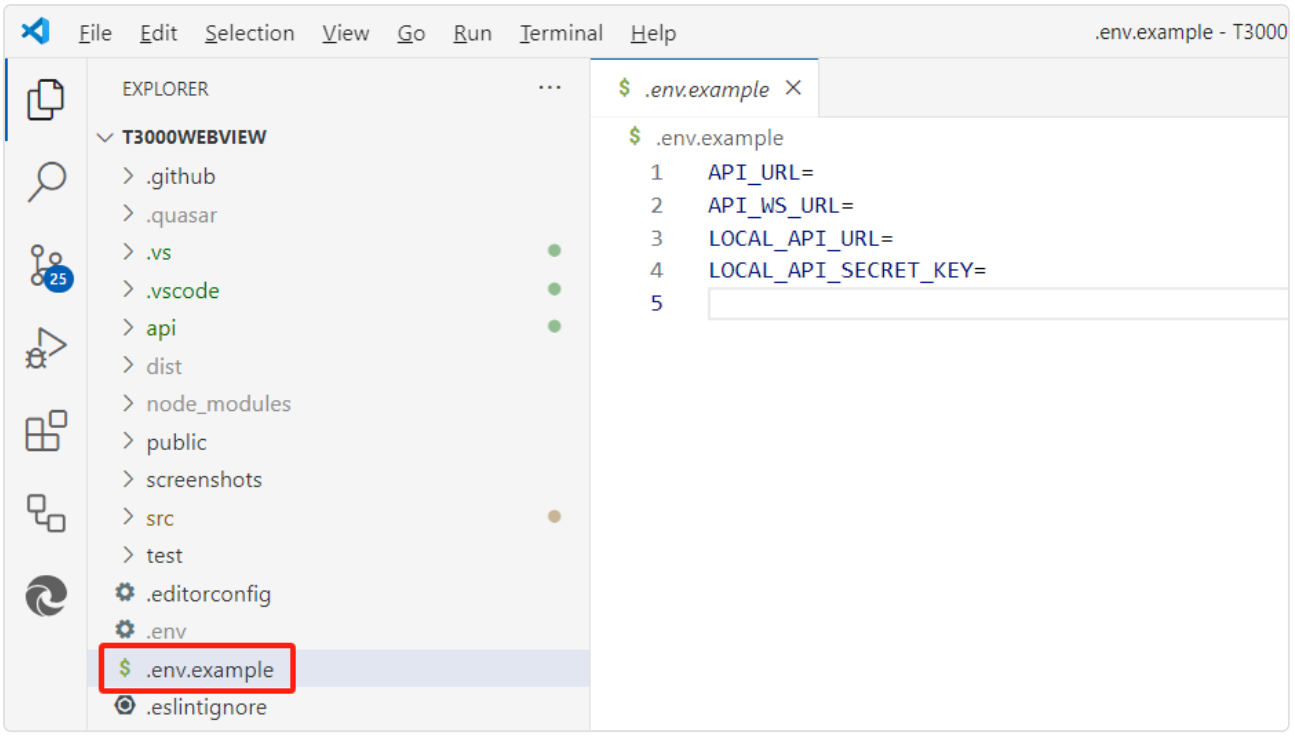
Debugging T3000Webview

1. clone the T3000Webview source code to local with below command.

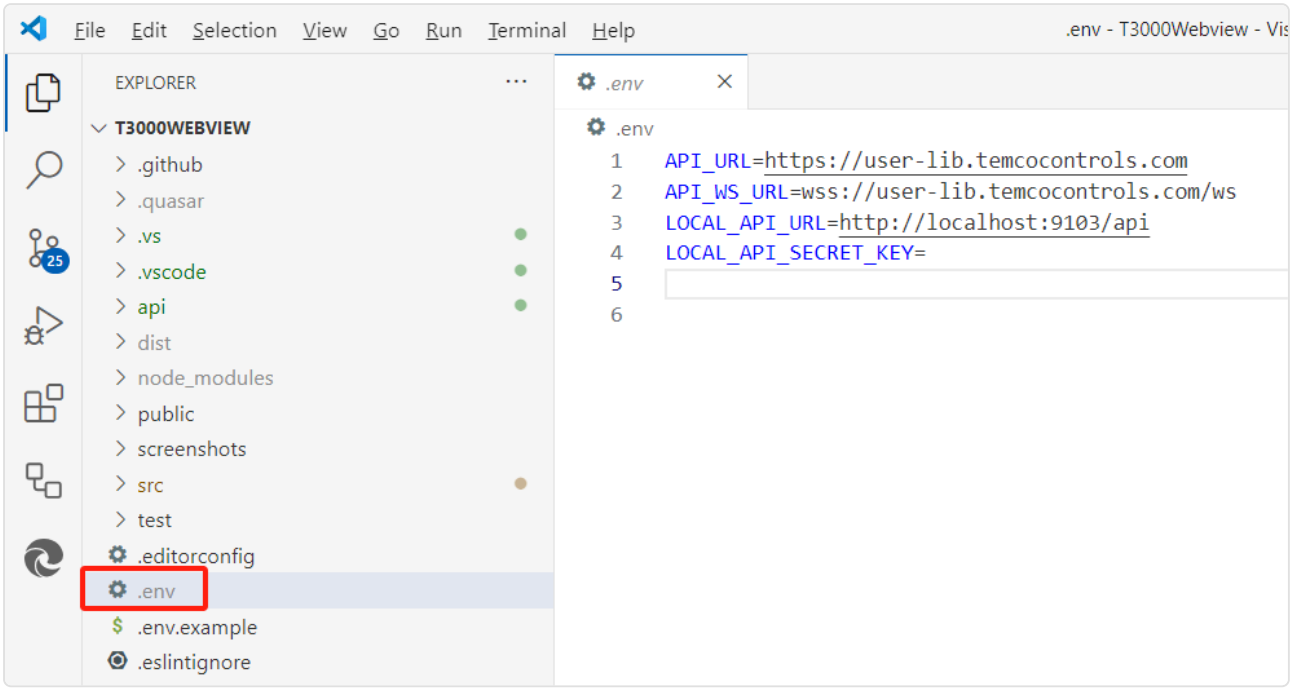
git clone https://github.com/temcocontrols/T3000Webview.git



2. open the source code via Visual Studio Code (or some other tool), find the .env.example file in the root folder “\T3000WEBVIEW”



3. add a new config file with name ".env" and copy the keys from ".env.example" and set the new values as below.



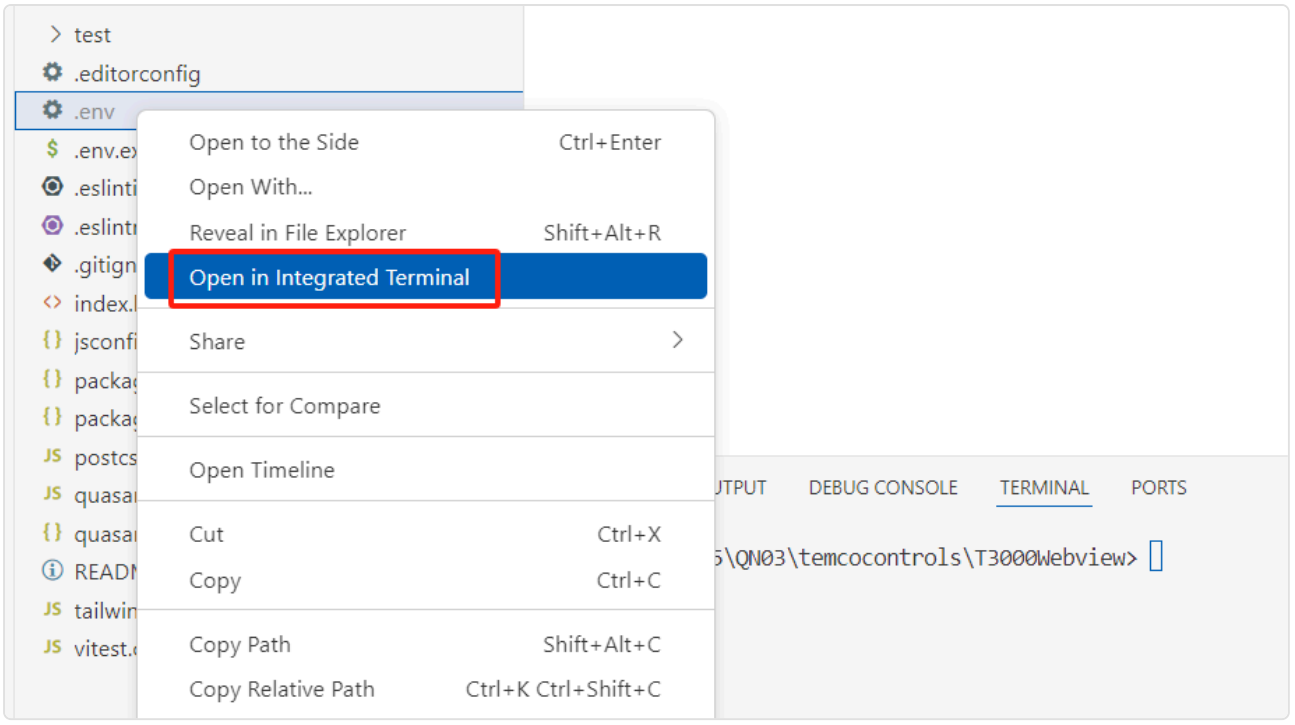
API_URL=https://user-lib.temcocontrols.com

API_WS_URL=wss://user-lib.temcocontrols.com/ws

LOCAL_API_URL=http://localhost:9103/api

LOCAL_API_SECRET_KEY=

4. right click to open the "Integrated Terminal".



5. install the packages with below command.

npm i



6. build the web project with below command.

npm run build

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS
PS D:\QN\1025\QN03\temcocontrols\T3000Webview> npm i

up to date in 2s

131 packages are looking for funding
  run `npm fund` for details
PS D:\QN\1025\QN03\temcocontrols\T3000Webview> npm run build

> t3-webview@0.8.1 build
> quasar build


 .d88888b.
d88P" "Y88b
888     888
888     888 888  888  8888b.  .d8888b   8888b.  888d888
888     888 888  888     "88b 88K          "88b 888P"
888 Y8b 888 888  888 .d888888 "Y8888b. .d888888 888
```

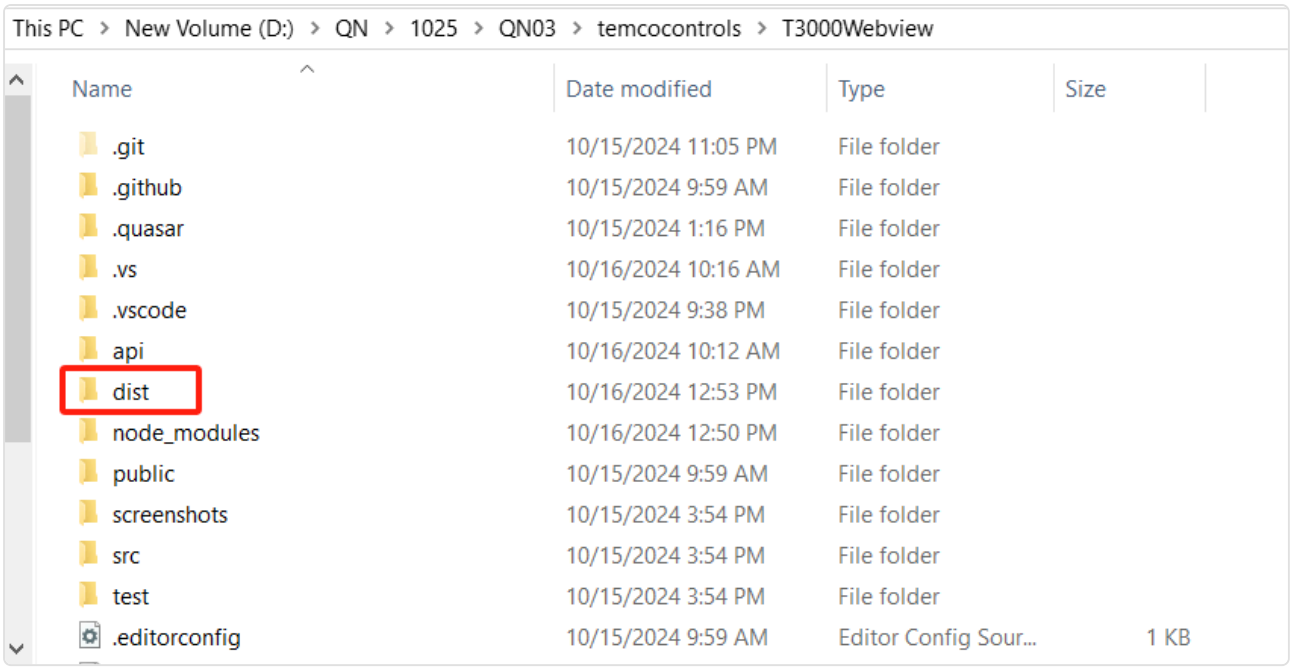
```
Build succeeded

Build mode.............. spa
Pkg quasar.............. v2.16.4
Pkg @quasar/app-vite... v1.9.3
Pkg vite................ v2.9.18
Debugging............... no
Publishing.............. no
Browser target......... es2019|edge88|firefox78|chrome87|safari13.1
========================
Output folder.......... D:\QN\1025\QN03\temcocontrols\T3000Webview\dist\spa

Tip: Built files are meant to be served over an HTTP server
     Opening index.html over file:// won't work

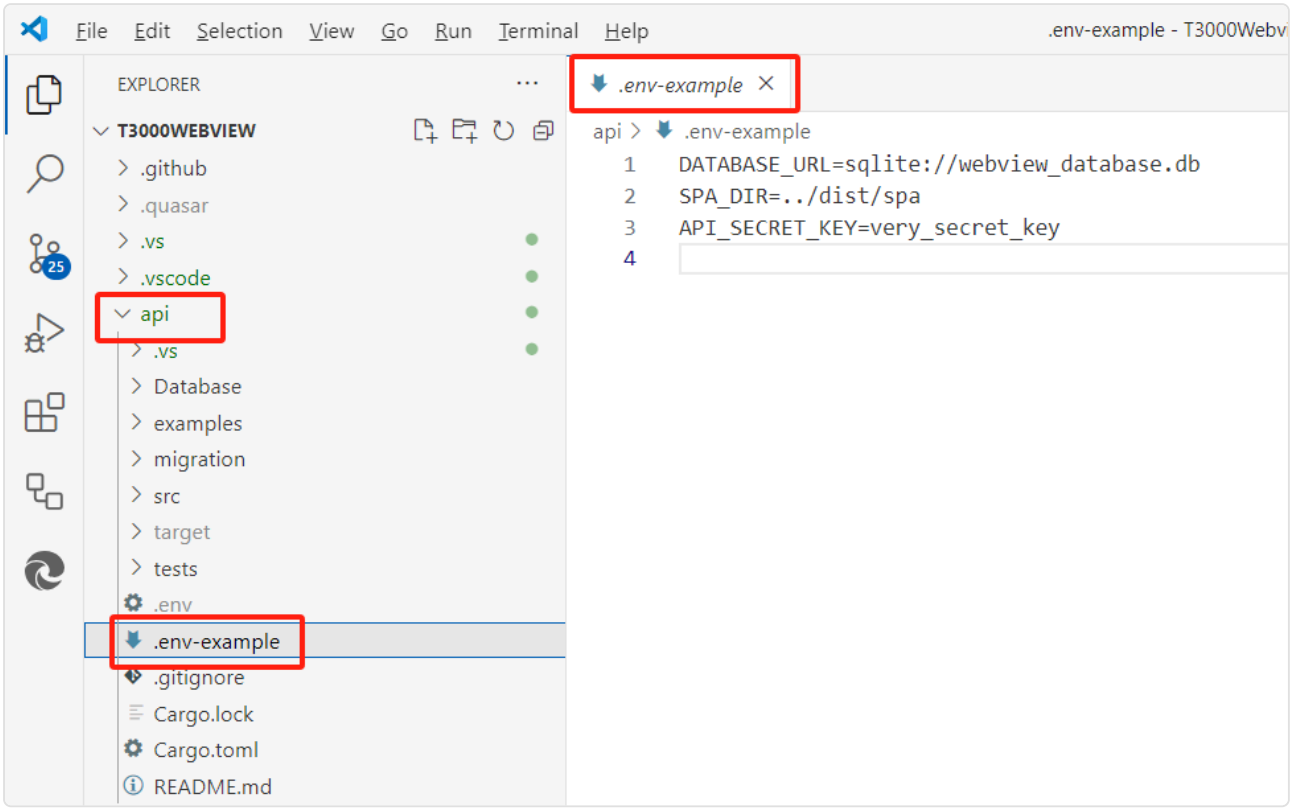
Tip: You can use "$ quasar serve" command to create a web server,
     both for testing or production. Type "$ quasar serve -h" for
     parameters. Also, an npm script (usually named "start") can
     be added for deployment environments.
     If you're using Vue Router "history" mode, don't forget to
     specify the "--history" parameter: "$ quasar serve --history"

PS D:\QN\1025\QN03\temcocontrols\T3000Webview>
```

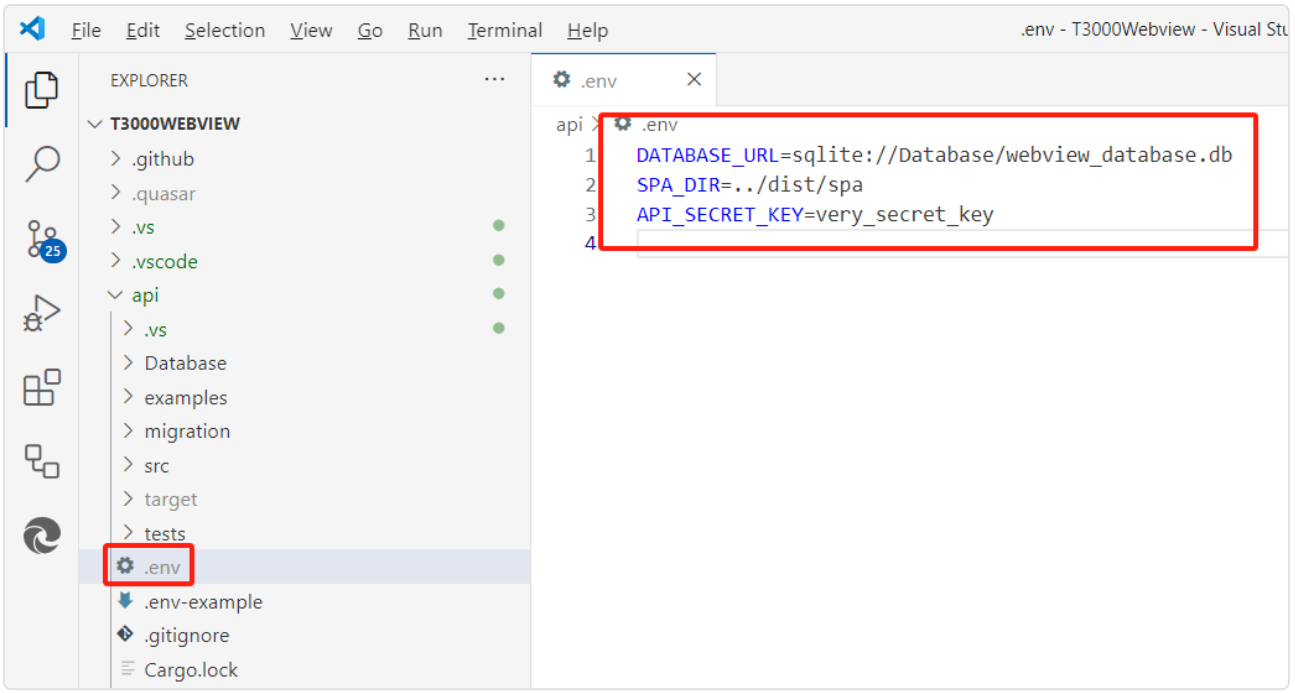


7. open the ".env-example" file under api folder. (full path should be \T3000Webview\api)

The api folder stores the source code of Rust back-end api



8. and a new config file under "\T3000Webview\api" with name ".env" and copy the keys from ".env-example", update the values as below.



DATABASE_URL=sqlite://Database/webview_database.db

SPA_DIR=../dist/spa

API_SECRET_KEY=very_secret_key

9. open the “Integrated Terminal” again, and cd into api folder



10. execute following command to start the rust api server.

cargo run --example run_server

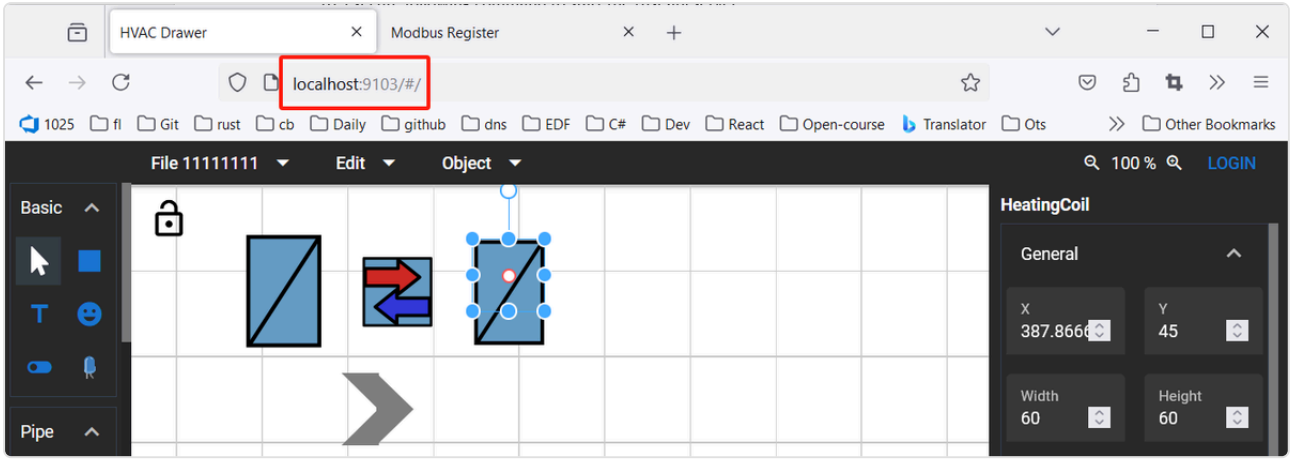
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

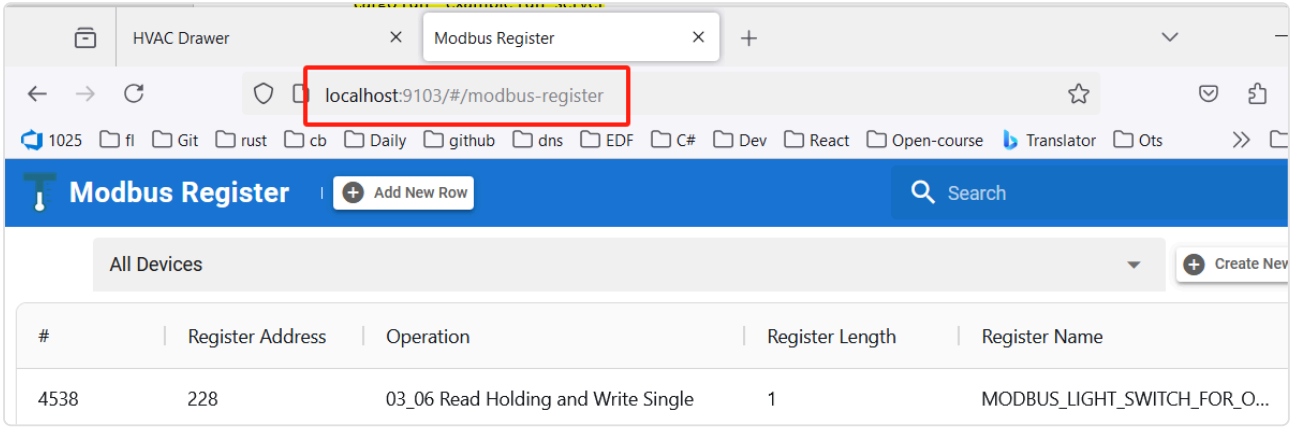
PS D:\QN\1025\QN03\temcocontrols\T3000Webview\api> cargo run --example run_server
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.31s
     Running `target\debug\examples\run_server.exe`
2024-10-16T05:06:16.122682Z  INFO sea_orm_migration::migrator: Applying all pending migrations
2024-10-16T05:06:16.125327Z  INFO sea_orm_migration::migrator: No pending migrations
->> LISTENING on Ok(0.0.0.0:9103)
```

check with below urls to make sure the server has been started and working well.

http://localhost:9103/#/

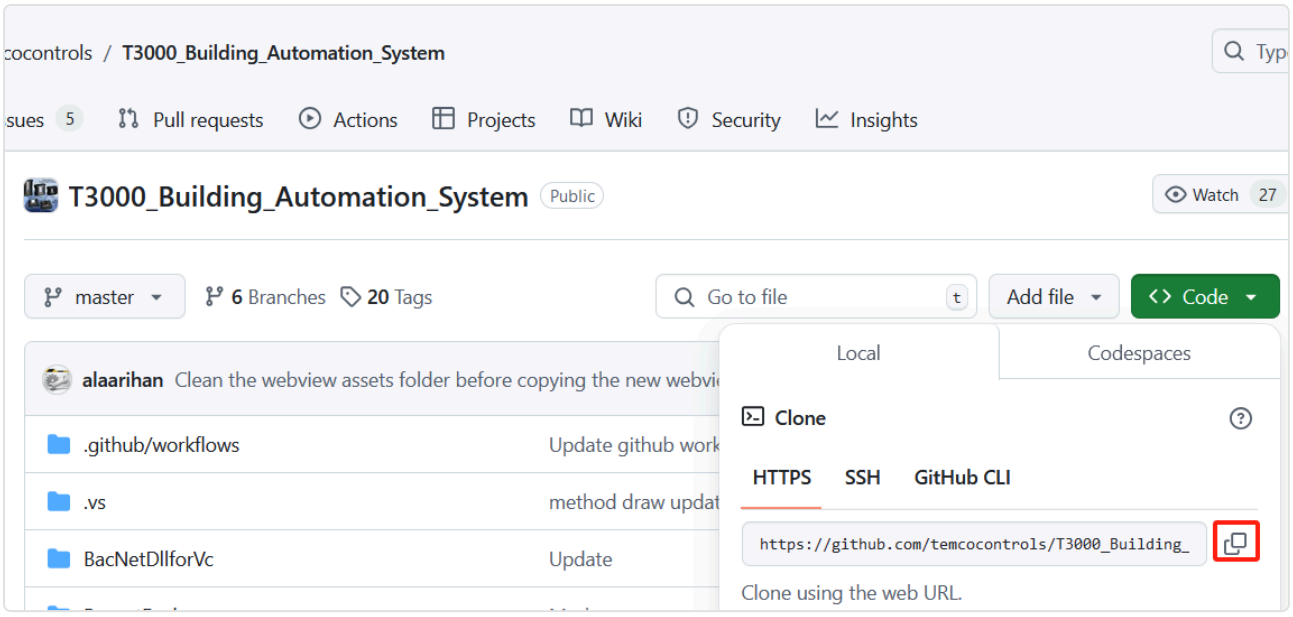
http://localhost:9103/#/modbus-register





Notes: if no need to debug it via T3000, can ignore below steps 11 & 12, can just use any web browser to do it.

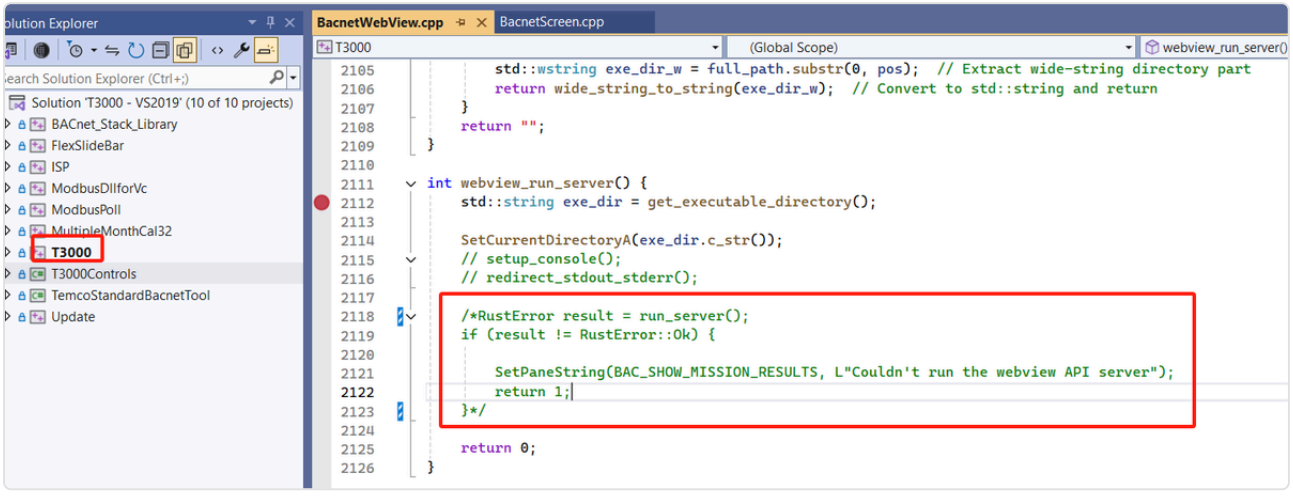
11. clone or update the source code of T3000_Buiding_Automation_System, make sure it's updated.



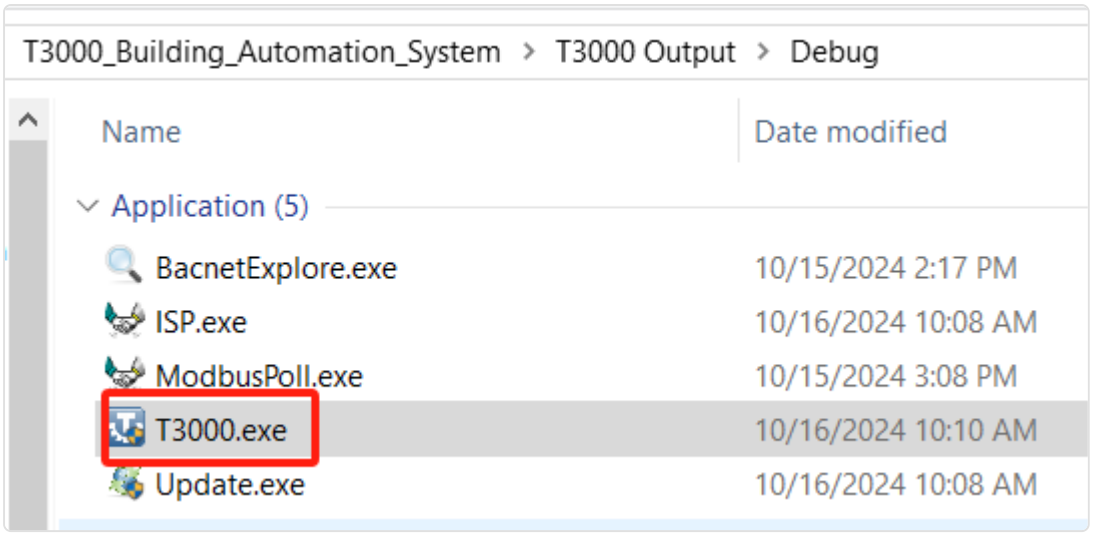
12. open the “T3000 - VS2019.sln” solution file with visual studio as administrator.

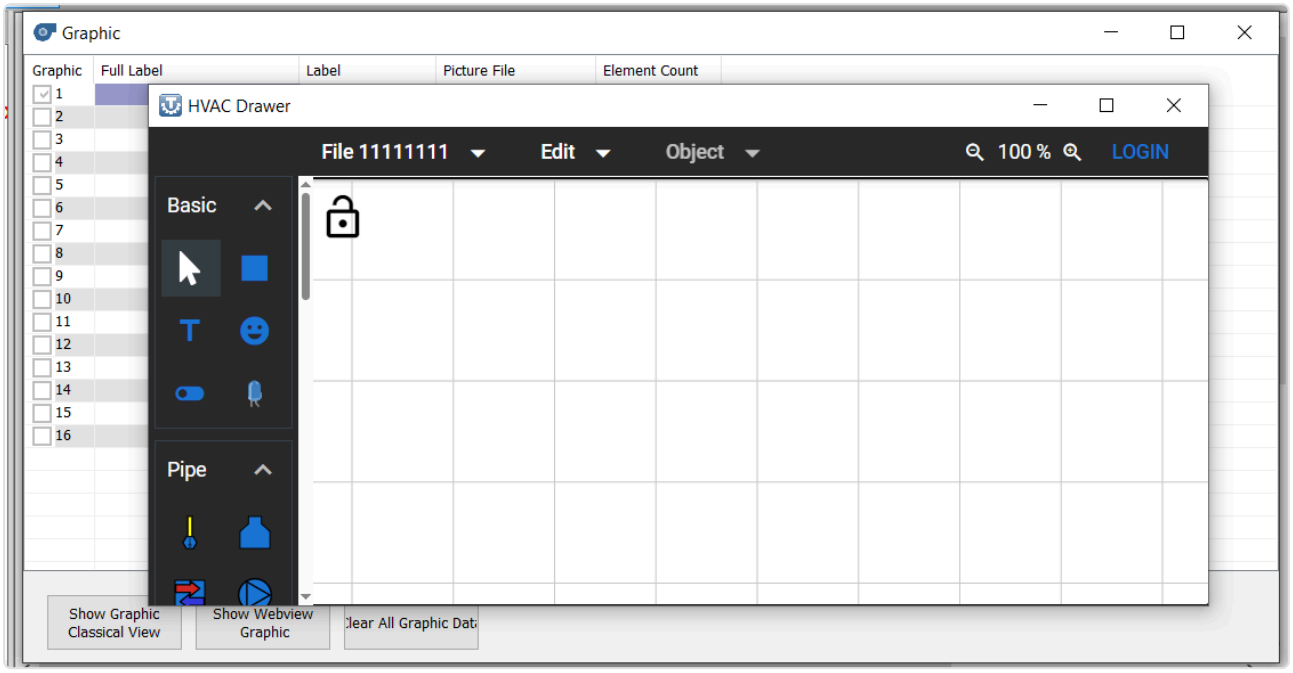
comment the code as shown below.

Make sure the T3000.exe does not start the rust server in background.

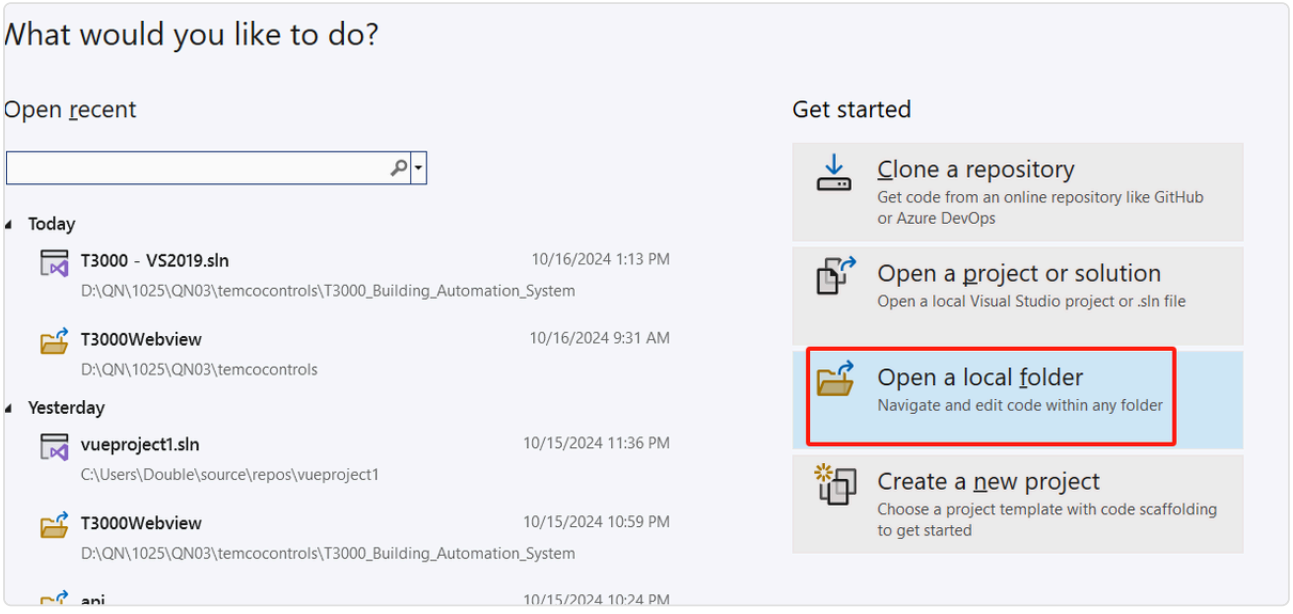


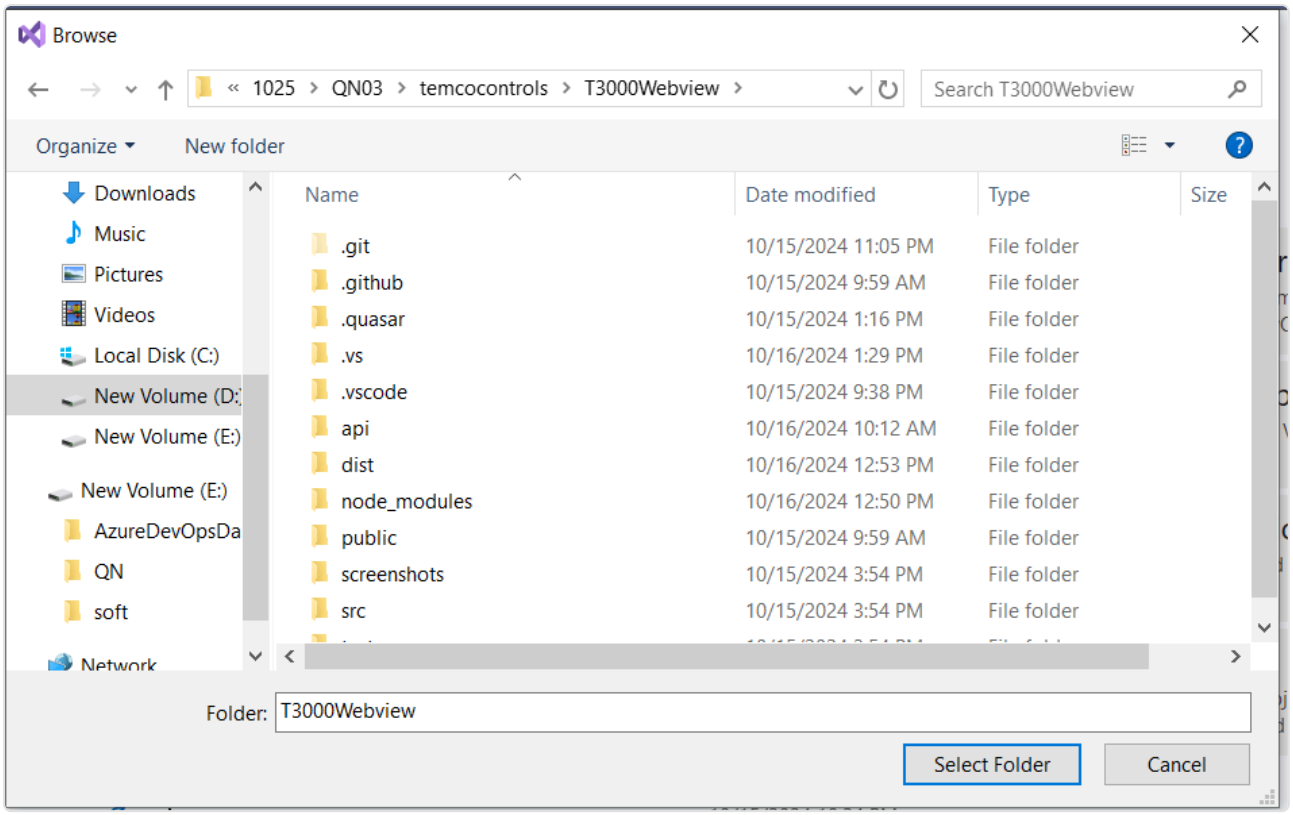
13. rebuild the solutions and run T3000.exe under T3000 Ouput\Debug, and check whether the “Show Webview Graphic” can access the rust server which was started by step 10



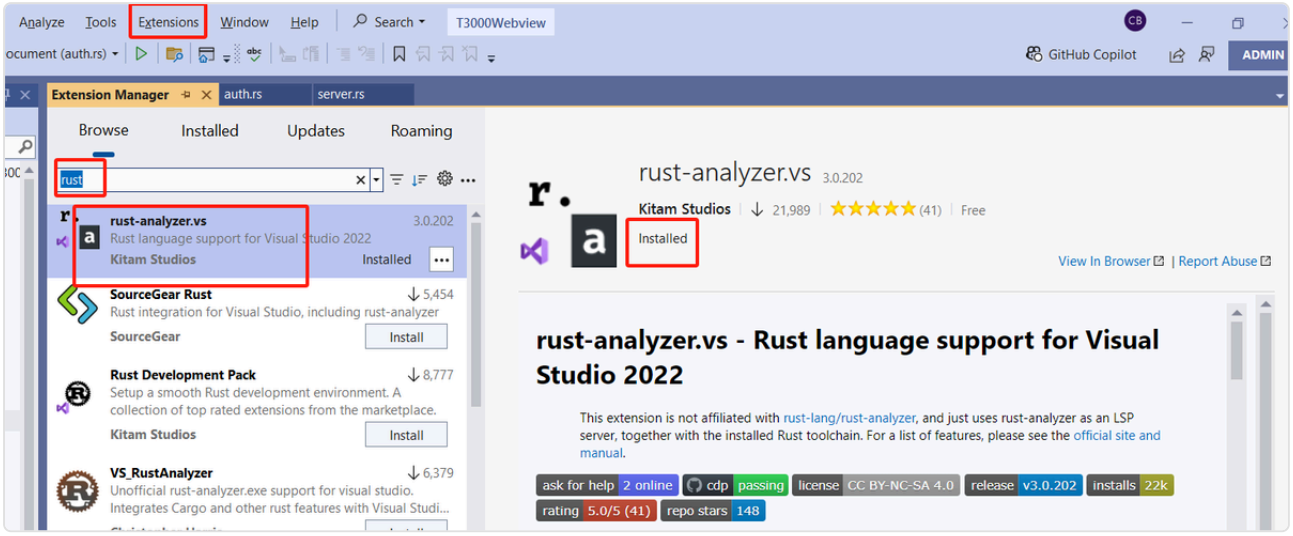


14. open the T3000Webview folder with Visual Studio 2022.

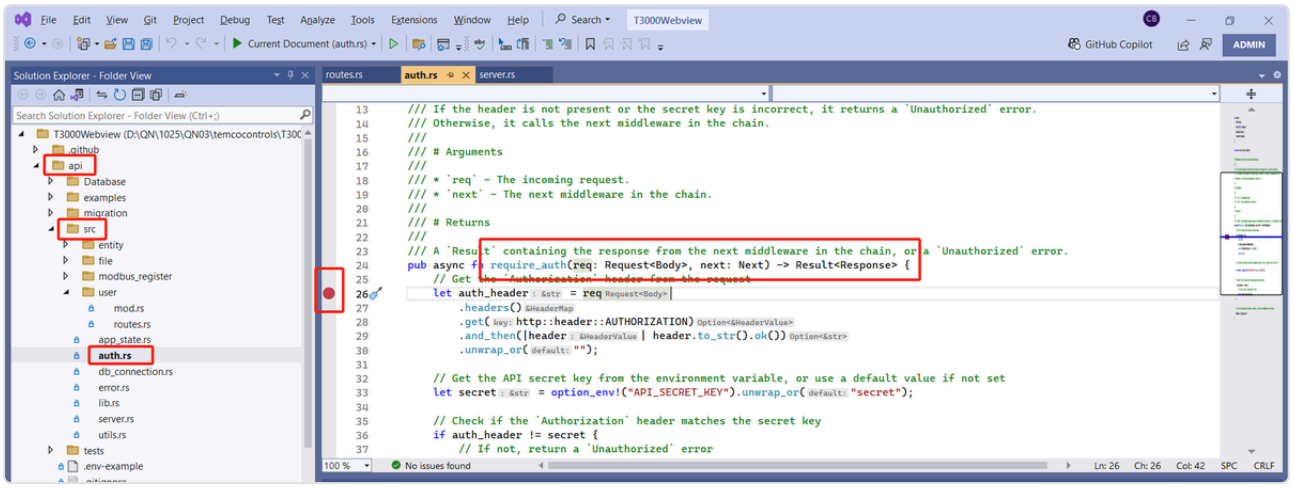




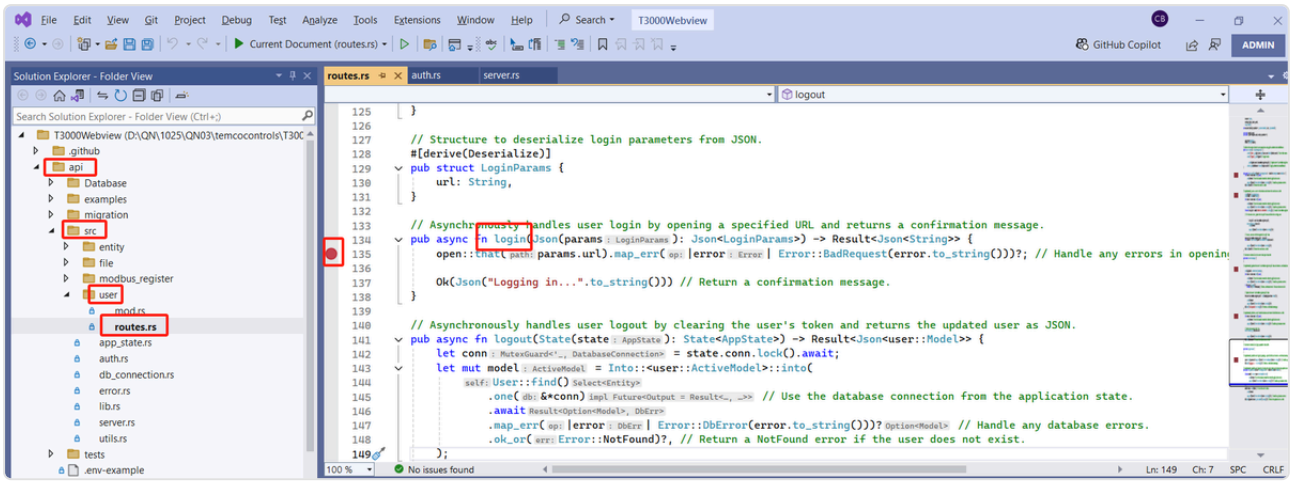
15. add rust-analyzer.vs extension to visual studio 2022.



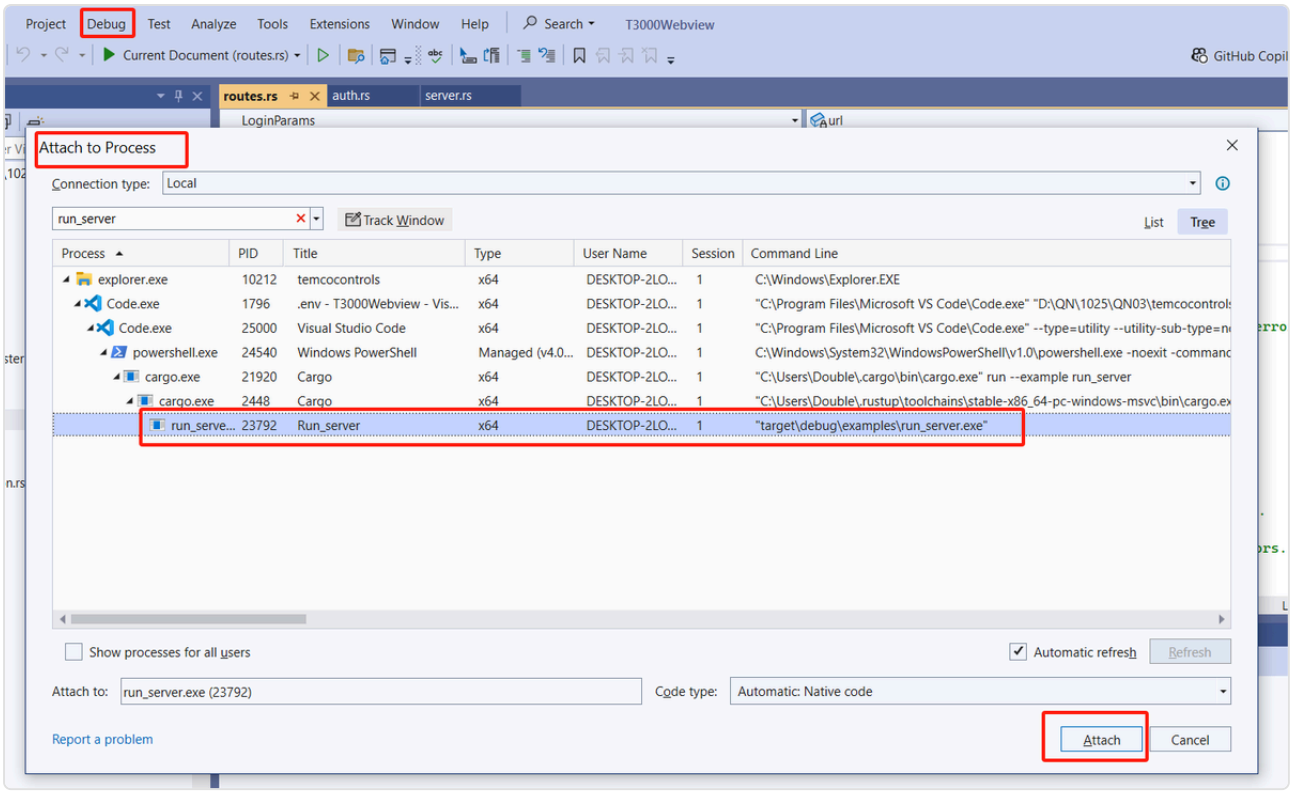
16. find auth.rs file under /api/src/auth.rs, and add a breakpoint for “require_auth” function.



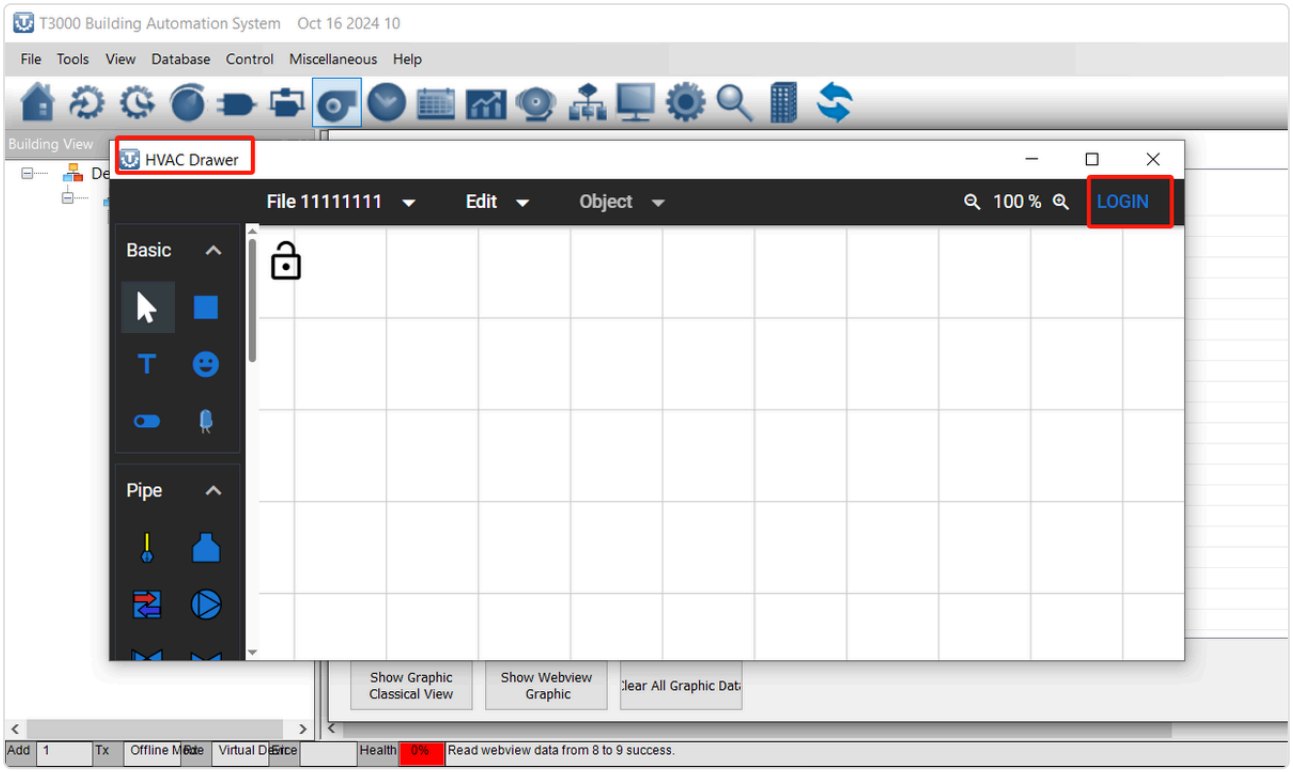
add another breakpoint for “login” function

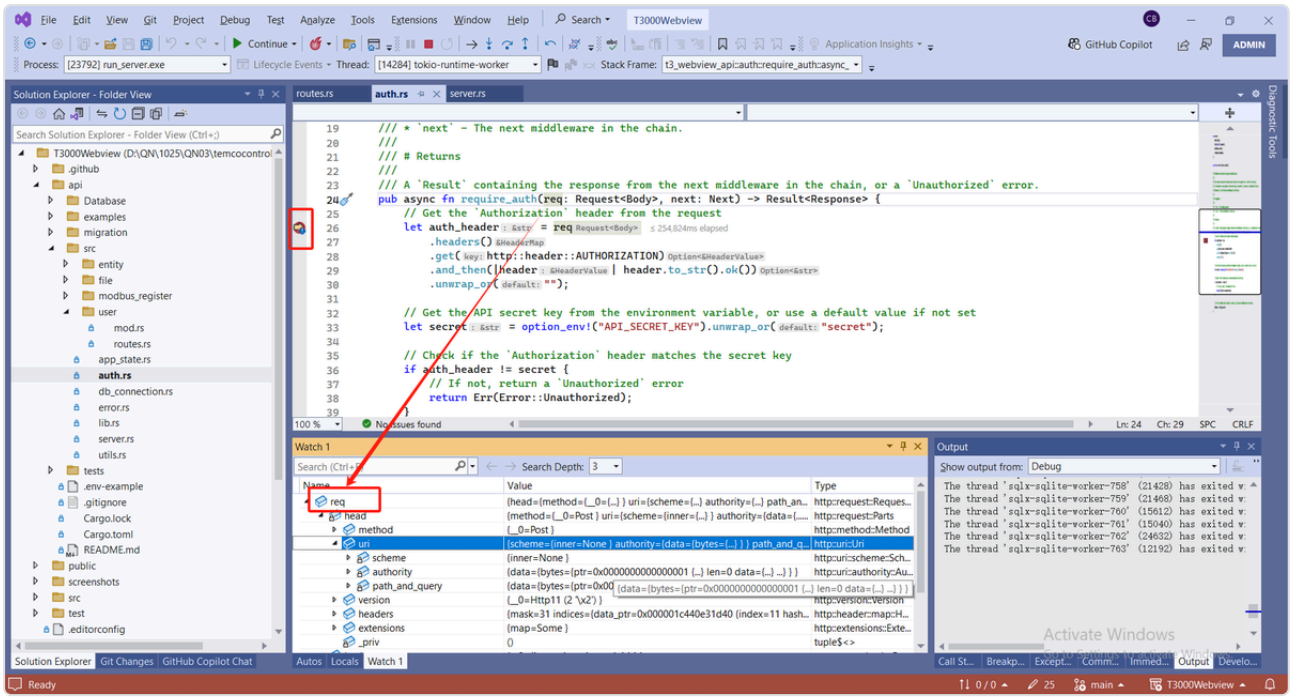


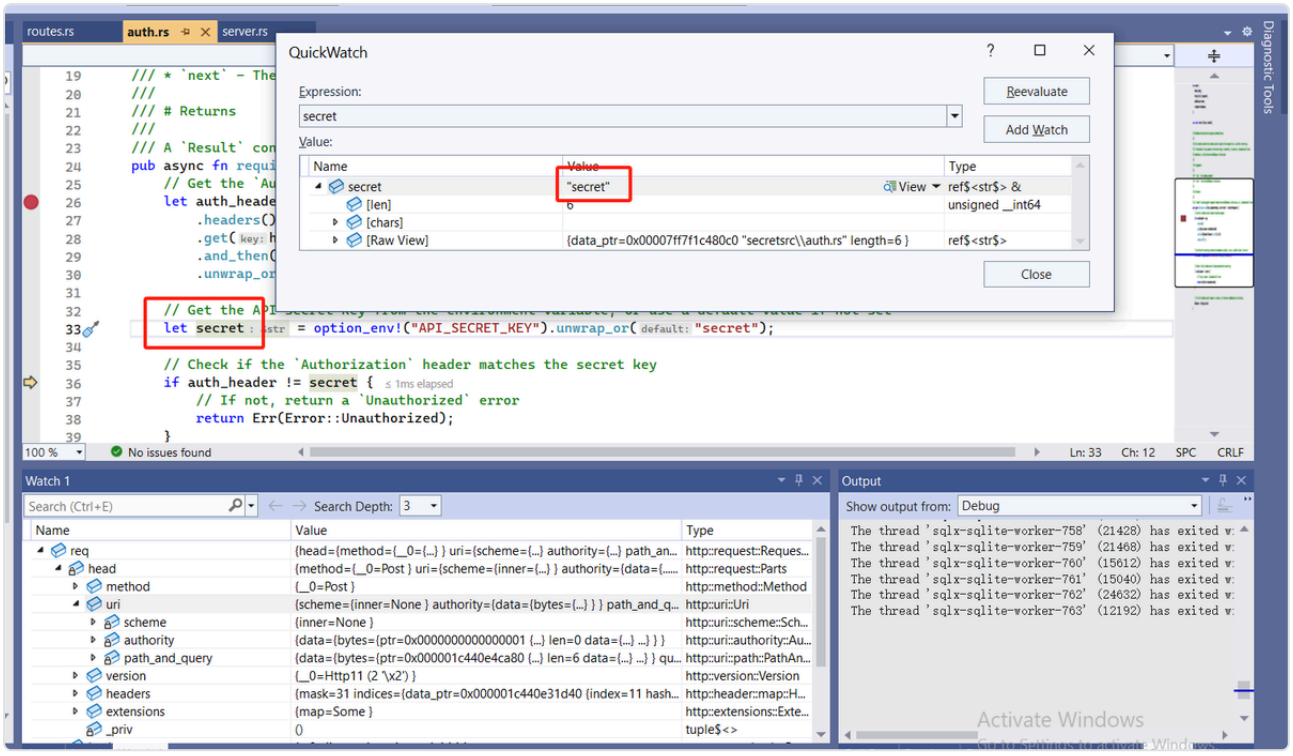
17. open the “Attach to Process” dialog, “Debug->Attach to Process”, and find the “run_server” process to attach.



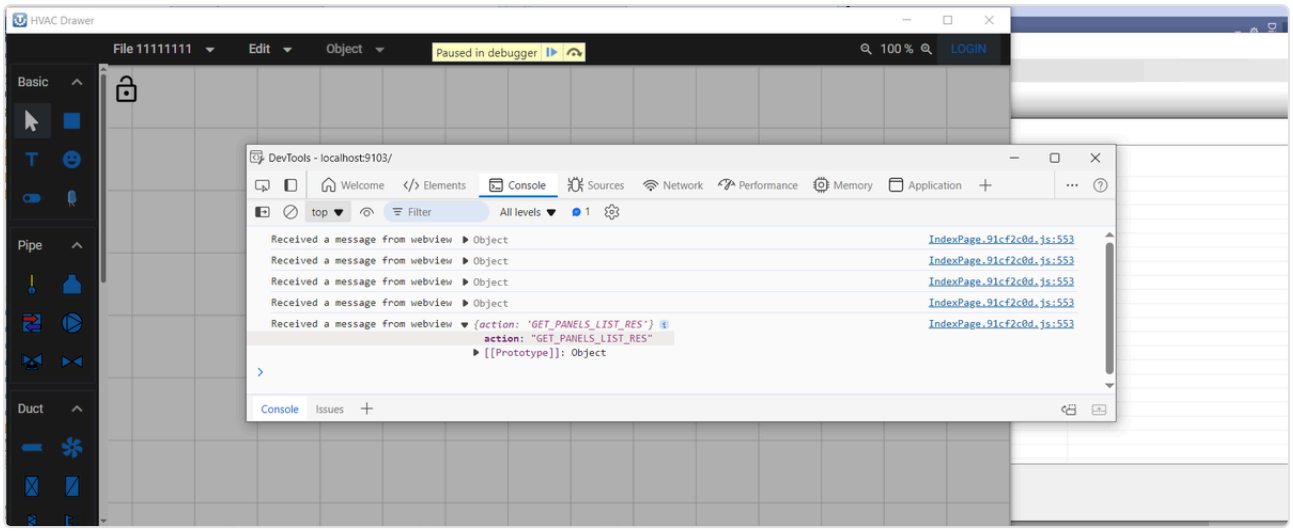
18. open “HVAC Drawer” and click the “LOGIN” button, it will redirect to the source code for debugging. You can check the details in visual studio.

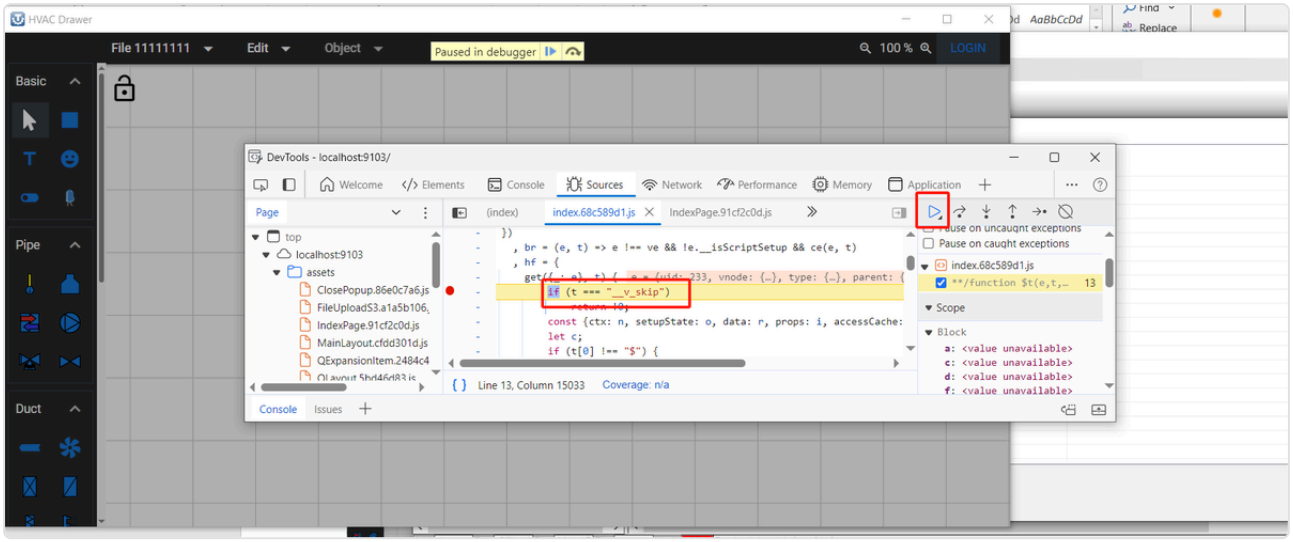






19. for the JavaScript and Html files, can open the Edge Dev Tools to check the log and some other details.


HVAC Drawer
File 11111111
Edit
Object
Paused in debugger
100 %
LOGIN
Basic
Pipe
Duct
DevTools - localhost:9103/
Welcome
Elements
Console
Sources
Network
Performance
Memory
Application
top
Filter
All levels
1
Received a message from webview
Object
IndexPage.91cf2c0d.js:553
{action: 'GET_PANELS_LIST_RES'}
action: "GET_PANELS_LIST_RES"
[[Prototype]]: Object
Console
Issues


HVAC Drawer
File 11111111
Edit
Object
Paused in debugger
100 %
LOGIN
Basic
Pipe
Duct
DevTools - localhost:9103/
Welcome
Elements
Console
Sources
Network
Performance
Memory
Application
Page
(index)
index.68c589d1.js
IndexPage.91cf2c0d.js
top
localhost:9103
assets
ClosePopup.86e0c7a6.js
FileUploadS3.a1a5b106.
IndexPage.91cf2c0d.js
MainLayout.cfdd301d.js
QExpansionItem.2484c4
if (t === "__v_skip")
const {ctx: n, setupState: o, data: r, props: i, accessCache:
let c;
if (t[0] !== "$") {
Line 13, Column 15033
Coverage: n/a
Pause on uncaught exceptions
Pause on caught exceptions
index.68c589d1.js
Scope
Block
a: <value unavailable>
c: <value unavailable>
d: <value unavailable>
Console
Issues